End-to-End Soundness of a Context-Sensitive Goblint-Style Analyzer in Isabelle/HOL

Manuel Lerchner ✉
Technical University of Munich, Germany

Abstract

Goblint-style abstract interpreters translate a program into side-effecting equations, with one unknown per program point and calling context, and solve them with a generic solver. The partial correctness of that solver has been verified in Isabelle/HOL, but the theorem does not connect the equations to the program. We mechanize this connection for a small imperative language with recursive procedures. We adapt the local traces of Schwarz et al. to procedure activations, because a return must resume exactly the activation that made the call. A context policy relates each activation trace to the calling contexts it may belong to. The development provides (i) a coverage contract of five local obligations over an arbitrary control-flow graph, which bounds the final store of every valid trace in every admitted context; (ii) a proof that every post-solution of the generated, context-routed equations (a valuation satisfying all of them) meets the contract when the context policy agrees with the routing, using the verified solver only through its certificate; and (iii) an end-to-end theorem from a small-step source semantics to the verdicts of the analyzer function exported to OCaml, for every configuration it offers. The theorem is per program point: every reached store is covered by the report and satisfies its definite verdicts. It is partial correctness, since solver termination is not proved, and the language has no pointers or heap.

2012 ACM Subject Classification Theory of computation → Program analysis; Theory of computation → Logic and verification; Software and its engineering → Formal software verification

Keywords and phrases abstract interpretation, interprocedural analysis, calling contexts, side-effecting constraint systems, verified static analysis, Isabelle/HOL

Related Version *Master's thesis and full formalization*: `https://github.com/ManuelLerchner/Voblint-Verification-Master-Thesis`

Supplementary Material *Software (Isabelle/HOL formalization and analyzer)*: `https://doi.org/10.5281/zenodo.23260696` [18]

1 Introduction

A context-sensitive static analyzer keeps a separate abstract state for each program point and calling context. In the program of Figure 1, `main` calls `bump(5)` and `bump(4)`, and `bump` returns its argument plus one. An interval analysis with one state per program point must describe both calls at the return of `bump`. It therefore cannot prove either check. With contexts that record the argument, it analyzes `bump` once per argument and proves both. Goblint, an abstract interpreter for C [36, 31], gains this precision by translating a program into a *side-effecting constraint system* [30, 1]. Its inequations relate unknowns indexed by program point and context, and the right-hand side of one unknown may also contribute to other unknowns. A generic top-down solver then solves the system. Goblint's soundness rests on tests: its issue tracker lists more than a hundred issues labeled as unsound since 2021, and a defect in any of these layers can turn a verdict false.

That solver has been verified. Stade et al. proved it partially correct in Isabelle/HOL [33], and Tilscher et al. extended the proof to side effects [34]. When the solver terminates, it returns a *post-solution*, a valuation that satisfies the inequations, on the unknowns it solved, a set closed under the reads of right-hand sides. Whether the equations describe the program lies outside that theorem. Closing the gap needs a concrete semantics in which

a per-context state has something to cover. Such semantics (Section 8) are classical on paper [32, 16, 14, 12]. To our knowledge, no mechanization connects a source semantics through such a context-indexed semantics to a side-effecting constraint system and its verified solver.

We mechanize this connection. The development, Voblint (verified Goblint), is written in Isabelle/HOL and contains no `sorry`. Its parts are the following.

- We adapt the local traces of Schwarz et al. [28] to procedure activations. A context policy, which may admit several contexts for one call, relates these activation traces to contexts. We prove that five local obligations on a cover bound every valid trace in every admitted context, over an arbitrary control-flow graph (Section 3).
- We prove that every post-solution of the generated, context-routed equations meets these obligations, provided the context policy agrees with the routing of the equations. The verified solver enters only through its certificate (Section 4).
- We prove an end-to-end theorem about the analyzer function that Isabelle exports to OCaml. Whenever it returns a report, every store a finite source execution reaches is covered by the report at its program point and satisfies the definite verdicts there, for every configuration (Section 5).
- We show that the theorem is not vacuous, that a domain obligation rejects a real Goblint defect, and how calling contexts change verdicts on recursive programs (Section 7).

The solver, side-effecting constraint systems, local traces, and Goblint's split between analyses and solvers are prior work. The context-indexed semantics is the structure of our proof. The end-to-end theorem is per program point. It is not a separate certificate for each context's row of the report (Section 5).

The scope is limited. The source language, VIMP, has integer variables, global variables, and recursive procedures with parameters and return values, but no pointers, arrays, or heap. The result is partial correctness: the analyzer answers only when its solver run terminates, and termination is not proved for every program. The parser and the report printer are unverified handwritten code. Isabelle's code generator, the OCaml toolchain, and a solver-tracing hook are trusted.

2 Overview: One Call Through the Pipeline

We follow the program of Figure 1 through the analyzer, with intervals and entry-state contexts. The source program is compiled to a procedure-aware control-flow graph (CFG). Each procedure p has an entry node $\mathsf{FunctionEntry}\ p$ and a result node $\mathsf{FunctionResult}\ p$. The graph g has two kinds of edges. An *intraprocedural* edge $(u, a, v) \in \mathit{intra}\ g$ carries an action a, such as an assignment or a guard. A *call* edge $(u, e, \mathsf{FunctionEntry}\ p, k) \in \mathit{calls}\ g$ leads from the call site u into p and names the continuation k at which the caller resumes. A predicate $\mathcal{G}$ tells which variables are global. A proved forward simulation relates every source execution to a graph execution with the same store (Section 5).

From the graph, the analyzer generates inequations over *local unknowns* (v, c) for a node v and a context c, and over *global unknowns* without a program point. At `a = bump(5)`, the analysis's *enter* operation binds `n` to $[5, 5]$, and this entry state is *routed* to the callee context $c_1 = [5, 5]$; the second call routes to $c_2 = [4, 4]$. Section 4 gives the equations of a call. Each activation of `bump` is analyzed in its own copy, and both checks are proved.

For these per-context states to mean something, the concrete semantics has to say which stores belong to c_1. Activation traces (Section 3) supply this. The trace of the activation `bump(5)` records its caller, the context policy admits it in c_1, and the *activation collecting*

```
fun bump(n) {
  return n + 1;
}

fun main() {
  a = bump(5);
  b = bump(4);
  __voblint_check(a == 6);
  __voblint_check(b == 5);
}
```

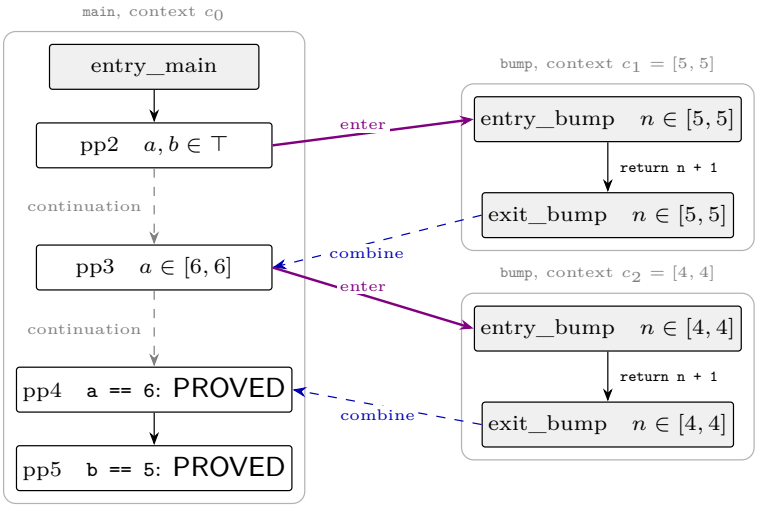


Figure 1 The running example. Left: the VIMP program. Right: its graph as the analyzer solves it with intervals and entry-state contexts. Each frame is one procedure in one context; `bump` gets one copy per entry state. Solid violet arrows are calls (enter), dashed blue arrows are returns (combine), and dashed gray arrows link a call site to its continuation. Both checks are PROVED. Without contexts, the single copy of `bump` joins both calls and both checks are UNKNOWN. States are analyzer output with the warrowing update rule for global unknowns (Section 4); the intermediate node of `bump`'s body and the unreachable node after `return` are omitted.

semantics $\mathcal{A}(v, c)$ collects the final stores of the traces that end at v and carry c. The *node collecting semantics* $\mathcal{C}(v)$ collects them regardless of context. A report *res* of the analyzer defines two sets at each node v. The report set $\mathcal{R}_{res}(v)$ contains the stores that some state of the report at v describes, in any context. The verdict set $\mathcal{V}_{res}(v)$ contains the stores in which every definite verdict at v holds. The soundness argument is a chain of inclusions at every node:

$$\text{reached stores} \subseteq \mathcal{C}(v) = \bigcup_c \mathcal{A}(v, c) \subseteq \mathcal{R}_{res}(v) \subseteq \mathcal{V}_{res}(v). \tag{1}$$

Each inclusion may add unreached stores, which costs precision, but none may lose a reached store. Contexts appear only in the middle of the chain; the end-to-end theorem (Section 5) states its outer inclusions.

3 Activation Traces and the Coverage Contract

Valid activation traces are adapted from the local traces of Schwarz et al. [28]. Context policies read a calling context off the concrete call, as the call-string approach of Sharir and Pnueli tags information with call history [32]. Their relational form is ours. The coverage contract and its soundness theorem are ours. Nothing here mentions an abstract domain, an equation, or a solver.

A context-sensitive analysis claims, for each node v and context c, a set of stores. Checking such a claim needs the stores that executions bring to v *in context* c. A graph execution with a plain frame stack does not record contexts: a frame holds only what is needed to resume the caller and disappears at the return. Frames could carry a ghost context (Section 6). We instead keep contexts out of the semantics. Grouping a run by activation makes the missing structure explicit: each call starts an activation attached to its caller, and each

return composes the finished callee back into that caller.

3.1 Activation traces

An *activation path* is the list of node–store pairs visited by one activation. An activation trace has one constructor for each way an activation begins or continues (`activation_trace`): Root π is the activation of `main`, Call t' π is a callee whose creator t' is frozen at the call site, and Resume t' t'' π continues t' past the call that produced the finished callee t''. The observers *path_of* t, *sink_node* t and *sink_store* t return the path and its last node and store, *caller_of* t returns *Some* of the creating caller, and *extend* t (v, s) appends a step to the path.

The datatype admits terms that no execution produces. The set $\mathcal{T}_{\mathcal{G},g,S}$ of *valid* traces of a graph g with initial stores S, where $entry_g$ is the entry node of `main`, is defined by four rules that mirror the rules of graph execution (`valid_activation_trace`):

$$\frac{s \in S}{\mathsf{Root}\ [(entry_g, s)] \in \mathcal{T}}$$

$$\frac{t \in \mathcal{T} \quad (sink_node\ t, a, v) \in intra\ g \quad s' \in step_a(sink_store\ t)}{extend\ t\ (v, s') \in \mathcal{T}}$$

$$\frac{t \in \mathcal{T} \quad (sink_node\ t, e, \mathsf{FunctionEntry}\ p, k) \in calls\ g}{\mathsf{Call}\ t\ [(\mathsf{FunctionEntry}\ p,\ enter_e(sink_store\ t))] \in \mathcal{T}}$$

$$\frac{\begin{array}{c} t'' \in \mathcal{T} \quad caller_of\ t'' = Some\ t' \quad sink_node\ t'' = \mathsf{FunctionResult}\ p \\ (sink_node\ t', e, \mathsf{FunctionEntry}\ p, k) \in calls\ g \end{array}}{\mathsf{Resume}\ t'\ t''\ \big(path_of\ t' \mathbin{@} [(k, combine_e(sink_store\ t', sink_store\ t''))]\big) \in \mathcal{T}}$$

Here $step_a$ is the concrete meaning of an action, $enter_e$ binds the callee's formals to the call's arguments and resets its locals, and $combine_e$ keeps the caller's locals, takes the callee's globals, and assigns the return value to the call's destination (`combine_collect`). A return resumes exactly the activation that created the callee, found through *caller_of*, without searching for a compatible frame. The return rule takes *some* call edge from the caller's call site into p. It does not require the edge through which the callee was entered. On compiled graphs every call site has exactly one call edge (`calls_source_unique`), so the two coincide.

Every finite source execution of a well-formed program yields a valid trace that ends at the corresponding node with the same store, and whose chain of callers represents the runtime frame stack (`source_run_has_activation_trace`). The final stores of valid traces, collected per node, form the node collecting semantics $\mathcal{C}_{\mathcal{G},g,S}(v)$ (`node_collect`).

In the local traces of Schwarz et al. [28], Root corresponds to an initial trace, Call to thread creation, and Resume to the binary rule for acquiring a lock, which incorporates a second trace (Section 8).

3.2 Calling contexts as a relation

An analysis chooses a callee's context at the call: with call strings of length k, the last k call sites; with entry-state contexts, the abstract values of the formals at entry. A *context policy* fixes this choice for every concrete call. It maps a call site u, a caller context c, a description of the call, the caller's store, and the entered store to the *set* of admitted callee contexts (`context_policy`). The result is a set because Goblint's enter may return several alternatives for one call, each routed separately. The set may be empty, for instance at a call

the analysis considers unreachable. The shipped analyses' enter returns a single alternative, so a call is admitted in at most one context per caller context. The relation matters for an enter with several alternatives: in a regression theory, a Sign analysis whose enter returns two overlapping alternatives (the plain entry state, and one that forgets the formals) admits one concrete call in two contexts (`ov_two_contexts_admitted`).

For a call edge e into p, we write $adm_{u,c}(e, s, s')$ for the set the policy returns for the call described by e and p, caller store s and entered store s'. We write $c' \in adm^{\exists}_{u,c}(p, s, s')$ when some call edge from u into p has entered store s' from s and the policy admits c' for it (`admits_call_context`). The contexts a trace carries are then defined inductively (`activation_context_rel`):

$$\frac{}{\mathsf{Root}\ \pi\ \text{carries}\ c_0} \qquad \frac{t'\ \text{carries}\ c}{\mathsf{Resume}\ t'\ t''\ \pi\ \text{carries}\ c}$$

$$\frac{t\ \text{carries}\ c \quad c' \in adm^{\exists}_{sink_node\, t,\, c}(p, sink_store\ t, s')}{\mathsf{Call}\ t\ ((\mathsf{FunctionEntry}\ p, s') \# \pi)\ \text{carries}\ c'}$$

That is, the root carries the initial context c_0, a callee carries every context admitted for its call from some context of its caller, and a resumed activation keeps the contexts it had before the call. Traces store no context, so the policy is a parameter of every theorem below. The activation collecting semantics collects the final stores of the valid traces that end at v and carry c (`activation_collect`):

$$\mathcal{A}_{\mathcal{G},adm,c_0,g,S}(v, c) \;=\; \{\, sink_store\ t \mid t \in \mathcal{T}_{\mathcal{G},g,S},\ sink_node\ t = v,\ t\ \text{carries}\ c \,\}.$$

In the running example, the final store of `bump(5)` lies in $\mathcal{A}(\mathsf{FunctionResult}\ \texttt{bump}, [5,5])$ and that of `bump(4)` in $\mathcal{A}(\mathsf{FunctionResult}\ \texttt{bump}, [4,4])$, whereas $\mathcal{C}$ at that node contains both.

3.3 The coverage contract

A *cover* assigns a set of stores to each node and context. Valid traces grow by four rules, so it suffices to check a cover against each rule once. The contract is the locale `activation_coverage`, Isabelle's mechanism for a named list of parameters and assumptions. It has one obligation per rule plus a totality condition. For every call edge $(u, e, \mathsf{FunctionEntry}\ p, k)$:

Init	$s \in S \implies s \in cover(entry_g, c_0)$
Intra	$(u, a, v) \in intra\ g,\ s \in cover(u, c),\ s' \in step_a(s) \implies s' \in cover(v, c)$
Call	$s \in cover(u, c),\ c' \in adm_{u,c}(e, s, enter_e(s))$ $\implies enter_e(s) \in cover(\mathsf{FunctionEntry}\ p, c')$
Return	$s \in cover(u, c),\ c' \in adm^{\exists}_{u,c}(p', s, s'),$ $t \in cover(\mathsf{FunctionResult}\ p, c') \implies combine_e(s, t) \in cover(k, c)$
Total	$s \in cover(u, c) \implies adm_{u,c}(e, s, enter_e(s)) \neq \emptyset$

Return is quantified over every procedure p' and entered store s' admitted at the call site, not only over the edge e it returns through. This matches the return rule, which does not tie the returning edge to the entering one. On a graph with one call edge per call site, $p' = p$ and the obligation reads as the familiar single-edge law. Total closes a gap that the other four leave. If a call from a covered store is admitted in no context, Call and Return hold vacuously and nothing places the store after the return at the continuation, although the resumed caller still carries its contexts there. Without Total, a cover that meets the

other four obligations can miss a reached store (`total_dropped_unsound`). TOTAL asks only about covered stores, because an entry-state policy admits no context for stores the analysis has excluded. Call strings and the unit policy, which admits one context for every activation, meet it for every cover (`call_context_total_on_of_fun`).

▶ **Theorem 1** (Context-indexed soundness, `activation_collect_sound`). *If a cover meets the five obligations of* `activation_coverage`*, then* $\mathcal{A}(v, c) \subseteq \mathit{cover}(v, c)$ *for every node* v *and context* c*.*

Proof idea. The proof is an induction over valid traces with the invariant that a trace carries some context and that its final store is covered in every context it carries. A single-trace invariant does not suffice for the return rule, which needs the invariant for the caller recovered through *caller_of*. The induction therefore carries the invariant along the whole chain of callers (`caller_chain_covered`, an instance of the induction principle `caller_chain_closure`). TOTAL is used twice. In the call case, it gives a new callee some context. In the return case, for each context c of the resumed caller, it re-admits the callee at some c' from c along the callee's own entry edge. The callee then carries c' (`activation_context_rel_caller_entryI`), so its exit store is covered at c', and RETURN composes the two at c. This is why RETURN quantifies over the admitted entry (p', s') rather than over the returning edge. Within the contract, TOTAL also gives every valid trace a context, so splitting by context loses nothing (`node_collect_eq_Union_activation_collect`):

$$\mathcal{C}(v) \;=\; \bigcup_c \mathcal{A}(v, c).$$

Section 4 shows that a solved equation system meets the contract.

4 From Equations to a Cover

The contract of Section 3 asks for a cover. This section constructs one from a solved equation system. The ingredients have different origins. The analysis interface mirrors Goblint's `Spec` [36, 31]. Side-effecting equations come from Seidl et al. and Apinis et al. [30, 1]. The solver and its certificate are those of Tilscher et al. [34]. The routing conditions, which connect the equations to the context policy, and their discharge for each policy are ours.

4.1 The analysis interface

An analysis describes the stores at a node by an abstract state d. Its meaning $\gamma(d, \rho)$ also depends on ρ, the solved values of the analysis's global unknowns. We suppress ρ when it plays no role. The analysis supplies the record `dg_spec`, the counterpart of Goblint's `Spec`: one transfer per kind of edge action, an enter operation, and two combine operations. A call needs two states. After the return, the caller's locals come from before the call, while the globals and the result come from the callee. Enter therefore returns a list of entry pairs (q, d): a resume state q describing the caller's store and an entry state d describing the store in which the callee starts. After the return, *combine_env* merges the callee's exit state into q, taking the callee's globals, and *combine_assign* then writes the return value to the call's destination. Together they mirror the concrete $\mathit{combine}_e$ of Section 3. A transfer may also publish to the analysis's global unknowns, and its output is read in the previous global values joined with what it publishes.

The local soundness laws of transfers and combine form the locale `analysis_contract`: the concretization is monotone, every transfer loses no successor store, and combine covers

$combine_e(s,t)$ whenever q covers s and the exit state covers t. Enter is not part of that locale. That enter returns a pair covering the caller's store and the entered store (`entry_pairs_cover`) is used by the routing conditions below, which each context policy discharges. A numeric domain proves only its abstract operations sound. The framework derives the transfers once for all domains, following Nipkow and Klein's generic interpreter [24, Sect. 13.5]. The shipped domains are Sign, Interval, Parity, Congruence, and their reduced product Int.

4.2 Context-indexed equations and routing

For every node v and context c there is a local unknown (v,c). Its right-hand side joins four sources (`routed_node_rhs_parts`):

- the initial state at the program entry;
- the transfer along each intraprocedural edge into v, in the same context c;
- the combined result of each call returning to v;
- at a procedure entry, the entry states published to it.

At a call, a *routing function route* chooses the callee context from the call site, the caller context, the entry state and the call. For a call edge $(u, e, \mathsf{FunctionEntry}\ p, k)$ and caller context c, the right-hand side of the continuation (k,c) contains, for every entry pair (q,d) that enter returns for $\sigma(u,c)$ (`routed_call_alternative_program`): $combine(q,\bot)$ if d is bottom, and otherwise, with $c' = route(u,c,d,e)$, the program

$$\text{publish } d \text{ to } seed(p,c'); \quad r \leftarrow \text{read } (\mathsf{FunctionResult}\ p,c'); \quad combine(q,r).$$

The entry equation of $(\mathsf{FunctionEntry}\ p,c')$ reads the global unknown $seed(p,c')$. In the running example, the call `bump(5)` in c_0 publishes $n \in [5,5]$ to $seed(\texttt{bump},[5,5])$, reads the result of `bump` in $[5,5]$, and combines it with the resume state of `main`. A recursive call needs no special treatment: its equation reads a result that depends on the caller's own unknowns, and the solver handles the cycle as it handles a loop.

The equations choose a callee context from an abstract state, while the concrete semantics chooses one from the concrete call through the policy adm. Two conditions make the choices agree. Both are assumptions of the locale `routed_context`.

Adequacy (`routing_adequate`). If adm admits c' for a call from a store that the caller's solved state describes, then enter produces a pair (q,d) routed to c'. The pair's resume state covers the caller's store, and its entry state covers the entered store. The callee's entry unknown in c' is solved.

Totality (`routing_total`). adm admits at least one context for every call from a store that the caller's solved state describes.

For the unit policy and for call strings, adm admits exactly the context the routing function computes from the call site and the caller context. Totality is then immediate, and adequacy reduces to entry coverage and the routed entry unknown being solved.

For entry-state contexts, the policy `routed_entry_context_rel` is defined from the solved valuation σ:

$$adm_{u,c}(e,s,s') \;=\; \{\, route(u,c,d,e) \mid (q,d) \in enter_e(\sigma(u,c)),\ s \in \gamma(q),\ s' \in \gamma(d) \,\}.$$

In words, a context is admitted when an entry pair of the solved caller state covers both concrete stores and is routed there. The definition is not circular. σ is fixed before the policy is applied, and the final guarantee concerns $\mathcal{C}$, which no policy changes. The policy only decides how $\mathcal{C}$ is split among contexts inside the proof. Section 8 compares this with a posteriori soundness [21]. A consequence matters for reading the result: under entry-state contexts, $\mathcal{A}(v,c)$ is relative to the analysis that produced σ.

Hypothesis	Reading
`analysis_contract`	transfers and combine are sound
`pp` (`post_bounded`)	the query is in W, and at each unknown of W, σ bounds the right-hand side and every side effect it publishes
`sg_cov`, `sg_uncov`	the cover is $\gamma(\sigma(v,c),\rho)$ on W and $\emptyset$ elsewhere
`fwd`	an intraprocedural edge from an unknown of W with a non-empty state leads into W
`comb_fwd`	a call's continuation is in W in the caller's context
`routing_adequate`, `routing_total`	the policy agrees with the routing
`resolve_sound`	callee resolution lists every procedure a covered store can call
`calls_unique`	each call site has one call edge (`calls_source_unique`)
`is_bot_sound`, `is_bot_bot`	a state classified as bottom describes no store, and $\bot$ is classified as bottom
`cmb_wf`, `extra_wf`	the generated call and seed contributions are well-formed programs over unknowns
`finC`, `finE`	the graph has finitely many edges
seed keys	seed unknowns differ from all other global unknowns

Table 1 Hypotheses of Theorem 2, stated over a set W of local unknowns: all assumptions of the locales `routed_context` and `dg_context_activation`. The theorem adds three premises: the program entry is in W in context c_0, the initial state describes the initial stores under initial global values ρ_0, and $\rho_0 \leq \rho$.

▶ **Theorem 2** (Routed collecting soundness, `activation_collect_dg_sound`)**.** *Under the hypotheses of Table 1, if* $(entry_g, c_0) \in W$*, the initial state describes the initial stores under initial global values* ρ_0*, and* $\rho_0 \leq \rho$*, then* $\mathcal{A}(v,c) \subseteq cover(v,c)$ *for all* v *and* c*, where* $cover(v,c) = \gamma(\sigma(v,c),\rho)$ *on* W *and* $\emptyset$ *elsewhere.*

The proof checks the five obligations of Theorem 1 for this cover. The initial state and $\rho_0 \leq \rho$ give Init. The edge transfers and `fwd` give Intra. Adequacy with entry coverage gives Call, adequacy with combine soundness gives Return, and totality gives Total. Both bounds of `pp` are used: a value reaches its target unknown either as an answer or as a published side effect, such as an entry state published to a seed. Return is quantified over every admitted call edge from the call site (Section 3), and `calls_unique` reduces it to the edge being returned through.

4.3 The solver certificate

Theorem 2 needs a set W of unknowns that contains the program entry, is closed under execution steps, and on which σ is bounded. The verified top-down solver [34] provides a different certificate, `part_post_solution`, for the query x it started on, its solved set V, and its result σ: the query is in V, V is closed under the reads of right-hand sides, and σ bounds both the right-hand side and every side effect of each solved unknown. The two certificates point in different directions. The solver works backward from its query, the result of `main`, while Theorem 2 follows executions forward from the entry. The bridge takes W to be the *live* part of V: the unknowns (u,c) such that u belongs to a procedure q whose result (FunctionResult q,c) is in V (`live_unknowns`). The compiler ends every body with a return edge, so every node of a procedure reaches the procedure's result through intraprocedural edges and call-to-continuation steps. The result's right-hand side reads its predecessors, so closure under reads puts all nodes of a procedure into V once its result is in V in that context. Calls carry this into callees. A continuation (k,c) in V reads the callee's result

(FunctionResult p, c') whenever the routed entry state is not bottom (`sol_vars_callee_result`), so the callee's nodes are live in c', its entry among them. For a bottom entry state, no store enters the callee and nothing is needed. `live_unknowns_cover` proves, from a well-formed program and a returned solve, that the live set contains the program entry and satisfies the closure hypotheses of Table 1. This is structural reachability in the graph. It holds also for an activation that never returns, because the argument follows the equations, not executions. Three sets are involved: the solver's solved set V, its live subset W, on which Theorem 2 is applied, and the states the report publishes, which are taken from V. Publishing more states than the proof uses is sound, because the report only has to contain the cover.

The rest of the proof never inspects the solver. The analysis locale takes the solver as a parameter and assumes four facts about it (`certified_solver`): a run in its domain yields `part_post_solution` and a finite solved set, and a run of its executable form that returns lies in the domain and agrees with the logical solver. These facts are proved for the vendored solver under each of its five update rules for global unknowns (`td_certified_solver`). An update rule decides how contributions to a global unknown accumulate: by join, by join per contributing right-hand side, or by *warrowing*, which combines widening and narrowing [34]. Replacing the solver would not touch the semantic layer.

5 The End-to-End Theorem

The previous sections bound the activation collecting semantics of a *graph*. Two links remain: from source executions to graph executions, and from the solved valuation to the report the user reads. The forward simulation follows the CompCert methodology [19]. The annotation that locates a source configuration at graph nodes, and the composition, are ours.

From source to graph. VIMP has a small-step semantics `pstep` over configurations (r, s, frs) of a residual command, a store, and a frame stack. A call pushes a frame holding the caller's store and destination and continues with the callee's body followed by a *Restore* marker; the marker pops the frame and combines the stores as in Section 3 (`pstep`, rules `Call` and `RestoreStep`). A call whose arity does not match, or whose callee is undeclared, is stuck; well-formedness excludes such programs. The compiler translates each procedure body by continuations into a procedure-aware CFG, and a forward simulation matches every source step by graph steps that reach the same store (`csim_star`). A source configuration does not determine a graph node, since its residual command can occur at several places, as in `if (b) {x = 1;} else {x = 1;}`. The proof therefore annotates every command of the program with its nodes. The relation $p, s_0 \vdash (r, s, \mathit{frs}) \rhd v$ (`at_node`) holds when some run of the annotated program from s_0 reaches an annotated configuration that erases to (r, s, frs), names v, and is related to v by the simulation. In the example, the configuration after either branch is observed at both branch nodes. It is a relation, and the theorem below quantifies over every node at which a configuration is observed. Combined with Section 3, every store a finite source execution reaches lies in $\mathcal{C}$ at a node the simulation relates to the run (`source_reaches_node_collect`), and on the annotated program this holds at every node where the run is observed (`csim_point_compile_prog`).

From valuation to report. The analyzer function `run_voblint` takes a configuration (the analyses to run, the update rule for global unknowns, the context policy, including the call-string length, and the placement of program globals) and a program. It rejects an invalid

configuration or an ill-formed program, and otherwise runs the executable solver and returns Analysed *res* when the solve returns a result (`run_voblint`). The report lists, per node, the solved states of every solved context, and classifies each check `__voblint_check(b)` as PROVED, REFUTED, UNKNOWN, or DEAD. As in Section 2, $\mathcal{R}_{res}(v)$ is the set of stores that some state at v describes, in any context, and $\mathcal{V}_{res}(v)$ the set of stores in which every PROVED check at v holds and every REFUTED check fails.

In the statement below, Π is the procedure table of p, and $\mathcal{G}, \Pi \vdash (c, s, frs) \rightarrow_p^* (c', s', frs')$ is a finite execution of `pstep` from residual command c, store s and frame stack *frs*. $\mathcal{C}_{\mathcal{G},g,S}\, v$ is the node collecting semantics of the compiled graph g from the initial stores S.

▶ **Theorem 3** (End-to-end soundness, `run_voblint_source_sound`).

theorem *run_voblint_source_sound*:
  fixes *p* :: *imp_prog* **and** *s0 s* :: *store*
  defines *G_def*: $\mathcal{G} \equiv$ *declared_global p*
    and *Pi_def*: $\Pi \equiv$ *prog_table p*
    and *g_def*: *g* $\equiv$ *prog_cfg p*
  assumes *s0*: *s0* $\in$ *cinit_stores* $\mathcal{G}$
    and *run*: $\mathcal{G}$, $\Pi \vdash$ (*main_body* Π, *s0*, []) $\rightarrow_p{}^*$ (*residual*, *s*, *frs*)
    and *ans*: *run_voblint config p* = *Analysed res*
  shows ($\exists$ *v*. *p*, *s0* $\vdash$ (*residual*, *s*, *frs*) $\triangleright$ *v*)
    $\wedge$ ($\forall$ *v*. *p*, *s0* $\vdash$ (*residual*, *s*, *frs*) $\triangleright$ *v* $\longrightarrow$
        s $\in \mathcal{C}_{\mathcal{G},g,cinit_stores\ \mathcal{G}}$ *v*
      $\wedge$ *s* $\in \mathcal{R}_{res}$ *v*
      $\wedge$ *s* $\in \mathcal{V}_{res}$ *v*)

The premises are an initial store, in which every global is zero as for C objects of static storage duration and locals are unconstrained; a finite source execution from the body of `main` with any stopping point, so nonterminating programs are covered through their prefixes; and a returned report. The conclusion has two halves: the reached configuration is observed at some node, and at every node where it is observed, its store is collected, covered by the report, and consistent with every definite verdict there. The contexts appear only inside the proof, through $\mathcal{C}(v) = \bigcup_c \mathcal{A}(v, c)$ and Theorem 2. The end-to-end theorem is therefore per program point. At the report level, the theorem about contexts (`run_voblint_contexts`, stated with the predicate `context_sound`) bounds each $\mathcal{A}(v, c)$ by $\mathcal{R}_{res}(v)$, the union over all contexts at v. No theorem about `run_voblint` certifies the row of an individual context. Theorem 2 bounds $\mathcal{A}(v, c)$ by the solved state for c, but the report is not proved to list that state as the row for c. Closing this gap is future work (Section 9).

No separate premise is needed for termination: `run_voblint` answers only when its executable solve returns, and a returned run lies in the solver's domain. None is needed for coverage, which `live_unknowns_cover` derives from the returned solve, or for well-formedness, since a malformed program does not yield Analysed. The configuration is a quantified argument, so the statement holds for every configuration the function accepts.

Meaning of verdicts. PROVED states that whenever a covered execution reaches the check, the condition holds; REFUTED that it fails (`run_voblint_check_sound`). Neither asserts that the check is reachable, and REFUTED is not a verified counterexample. DEAD is the reachability claim: nothing is collected at the check's node (`run_voblint_dead_check_unreached`). Analogous theorems cover arithmetic diagnostics: without a diagnostic at a node, no divisor leaving it is zero (`run_voblint_arithmetic_safe`), and an error means the divisor is zero there (`run_voblint_arithmetic_refuted`).

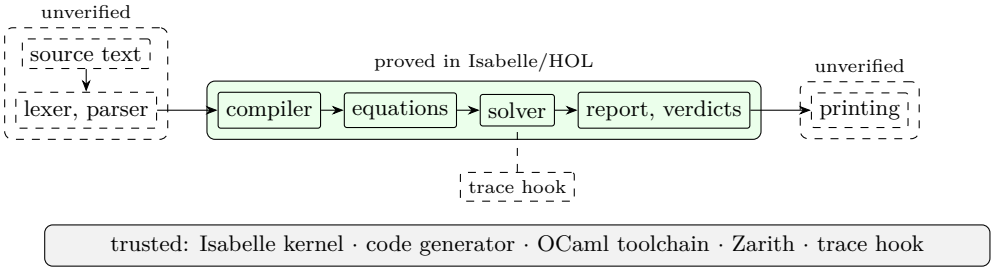


Figure 2 The trust boundary of the delivered analyzer, for programs whose solve terminates. Dashed boxes are handwritten code outside the proof. The trace hook is called by the solve, the routing and the report construction, and is assumed not to change their results.

Trusted components. Figure 2 separates what is proved from what is trusted. The theorem is about the constant that Isabelle's code generator [10] exports to OCaml and that the command-line tool calls. The lexer and parser, which construct the program, and the handwritten OCaml that prints the report lie outside the proof. The Isabelle kernel, the code generator, the OCaml toolchain, and the arbitrary-precision integer library are trusted. So is one handwritten OCaml function. For tracing, the exported code maps the logical constant `trace_event`, which is the unit, to a handwritten hook (`Solver_trace_hook.emit`) that the solve, the routing functions and the report construction call. The proofs treat each call as the unit, and the trusted base assumes that the hook does not change the results.

That `pstep` models the intended language is argued, not proved. Some of VIMP's choices differ from C. Integers are unbounded. Division by zero yields 0 and the remainder by zero yields the dividend. Expressions are pure and total, so `&&` and `||` have no evaluation order to model. A failed check does not stop the execution, so a verdict for one check does not depend on an earlier one. The locals of `main` start unconstrained, while a callee's locals start at 0. `min`, `max`, and the nondeterministic input are built-in calls, and `main` has no explicit `return`. A verdict therefore does not always hold for a C program with the same text.

6 Mechanization

Table 2 lists the size of the development [18] by layer. It counts the theories that the sessions list, and lemmas, theorems and corollaries as proved statements. The development reuses part of the verified solver of Tilscher et al. [34]. The solver is vendored with four changes made for this work and proposed upstream. They restore the build under Isabelle2025-2, remove from the widening and narrowing classes the stabilization assumptions that only termination proofs use, rename clashing names, and add facts such as `solve_dom_of_solve_c`. None changes the algorithm. Neither Voblint nor the imported solver theories contain `sorry` or `oops`. Code generation emits 13,109 lines of OCaml, and 4,922 lines of handwritten OCaml parse programs, print reports, and implement the trace hook. A full build takes several minutes on an Apple M4 Pro laptop.

Design choices and their alternatives. Besides the relational context policy (Section 3) and the solver as a parameter (Section 4), two choices shaped the proof. Only for the first was an alternative implemented and abandoned.

- *Activation traces as the soundness target.* A first prototype used whole-program executions with calls inlined. To account for one call, the proof built a separate execution from the callee's entry and spliced it into the caller's run. This handled single calls but did not

Layer	Contents	Theories	Lines	Proved
`Program_Model`	source semantics, CFG, compiler, traces, contract	27	9,996	466
`Abstract_Interpreter`	domain interfaces, D/G framework, routing, solver bridge	67	21,228	1,033
`Analyses`	Sign, Interval, Parity, Congruence, Int, relational witness	70	15,800	619
`Executable_Surface`	analyzer function, report, end-to-end theorems, export	15	5,222	180
`Examples`	non-vacuity, precision witnesses, regressions	34	7,231	397
Voblint total		213	59,477	2,695
Solver (imported)	Tilscher et al. [34]	7	9,533	168

Table 2 Size of the development by layer: lines of Isabelle/HOL including comments, and proved statements, which are `lemma`, `theorem` and `corollary` commands. The counts exclude one theory generated from the grammar manifest (`VIMP_Grammar_Generated`).

reflect the call stack and was hard to extend to recursion. The obvious alternative, which we did not implement, is the graph semantics itself, with a frame stack whose frames carry a ghost context. Its soundness proof would carry an invariant over the whole stack. Activation traces turn it into a fact about the caller chain, recovered through *caller_of*, and keep contexts out of the concrete semantics, so the policy can be a parameter.

Two-default executable states. Soundness is stated over total functions on variable names; the solver runs on a quotient type [13] of a finite override list with separate defaults for locals and zero-initialized globals (`exec_local_spec_sound`), refining the single default of Concrete Semantics [24, §13.6].

The contracts of Sections 3 and 4 and the certified solver are locales [2]; a domain is registered with one locale interpretation per context policy.

Feedback on the proof assistant. Totality and code generation cost effort that the tooling could reduce. Functions in Isabelle/HOL are total. A solver that may diverge is therefore defined with a domain predicate, next to an executable variant that returns only when the recursion finishes, and a returned run must be transported to the logical definition. A lemma added to the vendored solver proves this transport once (`solve_dom_of_solve_c`). Code generation also turns finite sets and maps into lists that every lookup scans, and CPU profiles attribute most of the solve time to these scans. The Containers library could replace them in the code setup alone, without touching the proofs; we leave this to future work.

7 Evaluation

A machine-checked theorem can hold for uninteresting reasons. Its premises may never hold together, its obligations may exclude no error a real analyzer makes, and an analyzer that answers UNKNOWN everywhere satisfies it as well (`unknown_everywhere_sound`). We therefore check that the premises can hold and that the obligations have force, and then measure what contexts and domains gain in precision on concrete programs. Each item states its kind of evidence. Facts about one fixed program are proved by `eval`, which runs generated code and therefore also trusts the code generator. Analyzer outputs are registered claims that `pixi run thesis-claims` re-checks; Tables 3 and 4 are generated from claims and from the committed profiling data.

Context policy	Contexts	`res1 == 2`		`res2 == 1`	
one context per procedure	2	UNKNOWN	$[1, 12]$	UNKNOWN	$[1, 12]$
call strings, length 1	4	UNKNOWN	$[1, 12]$	UNKNOWN	$[1, 12]$
call strings, length 2	6	PROVED	$[2, 2]$	PROVED	$[1, 1]$
entry states	6	PROVED	$[2, 2]$	PROVED	$[1, 1]$

Table 3 Callee contexts of `f` and `g`, verdicts and computed values for the two checks of the regression fixture `09-depth2_mutual_recursion_in_loop` (group `17-call-string`) under Interval, per context policy. Concretely, `res1` is always 2 and `res2` always 1. Generated from registered claims.

Non-vacuity (machine-checked, one program each). For the program of Figure 1, analyzed with Interval, entry-state contexts, and the warrowing update rule, a theory builds a source execution that returns from both calls. Evaluation shows that the analyzer answers with both checks PROVED, and `nv_source_certified` instantiates Theorem 3 with every premise discharged. Companion instances do the same for a single check (`nv_check_proved_sound`) and for a DEAD check (`nv_dead_unreached`). They show that the premises are satisfiable.

A Goblint defect the obligations reject (machine-checked). TOTAL is needed: `total_dropped_unsound` exhibits a cover that meets the other four obligations and misses a reached store. At the domain level, Goblint once claimed `c % 2 == 1` for $c \in \{-5, -7\}$, although C's truncating remainder gives -1 (issue 1156, fixed in [27]). The defect is in a domain, not in contexts. Every Voblint domain's remainder must meet `mod_sound` of the locale `sound_arith_ops` (the truncating remainder of concrete values lies in the concretization of the abstract one), which the shipped congruence remainder discharges by `congruence_mod_sound`. `prefix_congruence_mod_unsound` proves that every abstract remainder returning the constant 1 for $(1 + 2\mathbb{Z})$ mod 2 violates the obligation at -5, so the defective operator could not be registered.

Calling contexts on recursive programs (executable). Table 3 runs one regression program under every context policy. Mutually recursive procedures `f` and `g` return 2 and 1 at the bottom of the recursion. `main` calls `f(2)` and `g(2)` inside a loop of 200 iterations and checks both results. Each call chain is three call sites deep. The table also counts the copies of `f` and `g` the analysis creates. With one context per procedure, or with call strings of length 1, the activations of different depths share a state and both results widen to $[1, 12]$. With call strings of length 2 or with entry-state contexts, the depths stay apart and both checks are proved. The loop adds no contexts, because the callee contexts do not depend on the loop counter. For a related program, the theories prove that the solution of the routed equation system at call-string length 2 is strictly more precise at the entry of a callee than at length 1 (`sign_k2_strictly_more_precise_than_k1_at_g`). That theorem compares two solutions computed by `eval`. It concerns neither `run_voblint` nor call strings in general.

The thesis [18] also runs Voblint on the programs of Monniaux's tutorial on the flavors of abstract interpretation [22] and compares the verdicts with Goblint's.

Running time (empirical). Speed was not a goal: the generated code runs as Isabelle emits it, plus one proved code equation. The profiling run at repository revision `17fcb363d` analyzed the 380 regression fixtures that set no timeout of their own under 15 configurations: five domains times three context policies (none, call strings of length 1, entry states).

Context policy	Cases	Timeouts	Median	95 %	Max	Contexts
one context per procedure	1845	0	0.23	1.13	6	1
call strings, length 1	1845	0	0.24	1.47	11	23
entry states	1845	21	0.25	3.14	1106	302

Table 4 Solve time per context policy (milliseconds over successful cases), timeouts, and the largest number of contexts a successful case created. Generated from the committed profiling snapshot.

Eleven fixtures are ill-formed programs and are excluded, which leaves 369 fixtures and 5,535 cases. A case's time is the median of three repetitions on an Apple M4 Pro; a case that times out (30 s) or fails in any repetition counts as failed. Table 4 summarizes the successful cases per policy. All 21 timeouts are entry-state cases with Interval, Int or Congruence on seven recursive fixtures, where each recursion level creates a new context. One is `24-site-figures/precision/13-recursion_grows_warrow.vimp`: `f(x)` checks `x >= 0` and calls `f(x + 1)`. It is proved without contexts and does not terminate with entry-state contexts. Call strings of length 2 were not profiled. On generated programs of growing size, solve time grows with fitted exponents of 2.01 to 2.86 while solver evaluations grow about linearly (Section 6). These measurements cover small programs on one machine and imply no performance bound.

8 Related Work

Concrete semantics of calling contexts. Sharir and Pnueli distinguish the functional approach, which computes input–output relations of procedures, from the call-string approach, which tags information with the history of calls [32]. Voblint's call-string policy follows the second, and its entry-state policy, which indexes a callee's exit states by its entry state, resembles a functional approach tabulated per input. Knoop and Steffen handle the locals of recursive procedures with stacks of lattice elements, of which the maximal-fixpoint strategy needs only two [16]; Voblint's return has the shape of their return function [16, §4.1]. Reps et al. solve such problems precisely when facts are finite and transfer functions distributive [25]. Voblint requires neither and proves no precision result. Jeannet and Serwe derive interprocedural analyses by abstract interpretation of a standard stack-based operational semantics and combine the call-string and functional approaches [14]. Van Horn and Might derive sound analyses from abstract machines by bounding their stores [12]. These give contexts a concrete counterpart on paper. In the a posteriori soundness of Might and Manolios [21], the state abstraction is fixed in advance and a map from concrete to abstract contexts is chosen per run, after the analysis, from non-empty candidate sets. Voblint's entry-state policy instead reads a single relation off the solution, with Total in the role of non-emptiness (Section 4).

Verified analyzers. Verasco is a Coq-verified analyzer for most of C99, excluding recursion and dynamic allocation [15]. Its interpreter iterates over the structure of C#minor, reanalyzes a function at every call site, and raises an alarm at a possible recursive call. Blazy et al. check the candidate of an untrusted iterator with a verified checker [3]. Nipkow and Klein develop abstract interpretation for IMP in Isabelle/HOL [23, 24], and Voblint's non-relational domains follow their generic interpreter. Their language has no procedures. Franceschino et al. [9] and Cachera and Pichardie [6] verify executable analyzers for languages without procedures.

Michelland et al. make soundness proofs reusable by sharing one interpreter between the concrete and the abstract semantics [20]. Cachera et al. compute a context-insensitive dataflow analysis with a verified solver for ordinary inequations [5].

Contexts in mechanized semantics. Dabrowski and Pichardie instrument a concrete semantics with contexts through a module type whose call-context function computes a callee's context, and prove a context-sensitive points-to analysis sound in Coq [7]. This is the closest precedent for Voblint's context-indexed semantics. Their analyses are given only as constraints that no solver computes [7, §7]. Voblint's contexts are admitted by a relation, the totality condition makes the indexed collection exhaustive, and the indexing is connected to an executable verified solver. Trace partitioning indexes sets of traces by control history [26]; Rival and Mauborgne allow coverings in which one trace has several indices and present calling contexts as one such indexing. The activation collecting semantics is a covering of this relational kind. Among Isabelle developments, Lammich and Müller-Olm verify the soundness and precision of a conflict analysis for programs with procedures and threads [17], Wasserrab verifies an interprocedural slicer [37], and Breitner formalizes Shivers' control-flow analysis for functional programs [4]. None of these entries claims an extracted analyzer or represents an activation with its caller chain.

Verified solvers. Hofmann et al. verify a local generic solver in Coq [11], and de Vilhena et al. verify one that discovers dependencies dynamically in Iris [8]. Stade et al. prove the top-down solver partially correct in Isabelle/HOL [33], and Tilscher et al. extend it to side effects with a generic interface for update rules [34]. Voblint vendors the latter. Their termination result for top-down solvers without side effects assumes a finite type of unknowns [35], which Voblint's node and context types are not.

Goblint and local traces. Side-effecting constraint systems [30, 1] and the split between analyses and solvers are Goblint's architecture [36, 31], and Voblint's `dg_spec` follows its `Spec` interface. No theorem transfers to Goblint's OCaml implementation. Schwarz et al. prove Goblint's thread-modular analyses sound against local traces on paper [28, 29]; activation traces adapt the construction, but no concurrency result transfers.

Differences from prior work. Voblint inherits its solver, the side-effecting constraint systems, the analysis interface, the shape of its traces, and the distinction between call-string and functional contexts. Voblint adds their mechanized connection, ending in a theorem about the exported analyzer; we know of no prior mechanization of it. Dabrowski and Pichardie [7] have contexts in the semantics but no solver. Verasco [15] covers a far larger language but reanalyzes functions per call site, raises an alarm at recursion, and iterates structurally instead of using a generic solver.

9 Conclusion

We have mechanized the connection between a source semantics and a Goblint-style, context-sensitive, side-effecting equation system solved by a verified solver, with activation traces as the concrete counterpart of contexts. The theorem is a first step. The result is partial correctness, VIMP is scalar, the theorem is per program point, and precision is shown only by witnesses for single programs.

Per-context certification. Theorem 2 bounds each context by its solved state, but the report-level theorem keeps only the union over contexts. Proving that the report's row for a context covers $\mathcal{A}(v, c)$ would certify the per-context states.

Memory. The contract refers to the language only through a few step functions, so features such as machine integers leave it intact. The trace semantics, however, assumes that a callee cannot modify its caller's locals. Pointers into stack frames and threads break this assumption; a heap without such pointers needs only a richer store model.

Termination. Known termination results for top-down solvers assume finitely many unknowns [35]. Entry-state contexts need not be finite, and all timeouts in Section 7 come from them. Goblint bounds them with optional widening on contexts (`ana.context.widen`) or a context gas; a general result would need such a bound in the routing.

Data availability. The Isabelle/HOL development and the regression corpus are public on GitHub, and an earlier version is archived on Zenodo [18]. That archive lacks the vendored solver, the claims and profiling data behind Tables 3 and 4, and the scripts that render the paper's tables; they will be released together with this paper. The vendored solver [34] is not yet public; it is planned for publication in the Archive of Formal Proofs. Until then the theories cannot be built from public sources alone.

Declaration about GenAI/LLM use. Generative AI tools (Anthropic's Claude, OpenAI's ChatGPT and Codex, and Cursor) were used substantially in this work and contributed a large share of the implementation-level material under the author's supervision. In the Isabelle development, they generated many proofs once definitions and interfaces had stabilized, carried out refactorings across theories, and turned tactic-style proofs into structured Isar. They also assisted with design: they helped adapt the local traces of Schwarz et al. to procedure activations, and the requirements on activation traces were refined in collaboration with them. Agents wrote much of the surrounding tooling, including the command-line interface, the regression corpus, and the consistency checks. The author chose the research direction, selected and revised the designs, and approved the definitions and theorem statements; Isabelle checked every proof. The tools also assisted with literature search and with drafting and revising this text and its figures, which the author reviewed. Grammarly was used for spelling and grammar.

References

1 Kalmer Apinis, Helmut Seidl, and Vesal Vojdani. Side-effecting constraint systems: A swiss army knife for program analysis. In *Programming Languages and Systems (APLAS 2012)*, volume 7705 of *LNCS*, pages 157–172. Springer, 2012. doi:10.1007/978-3-642-35182-2_12.

2 Clemens Ballarin. Locales: A module system for mathematical theories. *Journal of Automated Reasoning*, 52(2):123–153, 2014. doi:10.1007/s10817-013-9284-7.

3 Sandrine Blazy, Vincent Laporte, André Maroneze, and David Pichardie. Formal verification of a C value analysis based on abstract interpretation. In *Static Analysis*, pages 324–344, 2013. doi:10.1007/978-3-642-38856-9_18.

4 Joachim Breitner. Shivers' control flow analysis. *Archive of Formal Proofs*, November 2010. URL: https://isa-afp.org/entries/Shivers-CFA.html.

5 David Cachera, Thomas Jensen, David Pichardie, and Vlad Rusu. Extracting a data flow analyser in constructive logic. *Theoretical Computer Science*, 342(1):56–78, 2005. doi:10.1016/j.tcs.2005.06.004.

6 David Cachera and David Pichardie. A certified denotational abstract interpreter. In *Interactive Theorem Proving (ITP 2010)*, LNCS, pages 9–24. Springer, 2010. doi:10.1007/978-3-642-14052-5_3.

7 Frédéric Dabrowski and David Pichardie. A certified data race analysis for a Java-like language. In *Theorem Proving in Higher Order Logics (TPHOLs 2009)*, LNCS, pages 212–227, 2009. doi:10.1007/978-3-642-03359-9_16.

8 Paulo Emílio de Vilhena, François Pottier, and Jacques-Henri Jourdan. Spy game: Verifying a local generic solver in Iris. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–28, 2020. doi:10.1145/3371101.

9 Lucas Franceschino, David Pichardie, and Jean-Pierre Talpin. Verified functional programming of an abstract interpreter. In *Static Analysis (SAS 2021)*, LNCS, pages 124–143. Springer, 2021. Hosted browser version: https://w95psp.github.io/verified-abstract-interpreter. doi:10.1007/978-3-030-88806-0_6.

10 Florian Haftmann and Tobias Nipkow. Code generation via higher-order rewrite systems. In *Functional and Logic Programming (FLOPS 2010)*, volume 6009 of *LNCS*, pages 103–117. Springer, 2010. doi:10.1007/978-3-642-12251-4_9.

11 Martin Hofmann, Aleksandr Karbyshev, and Helmut Seidl. Verifying a local generic solver in Coq. In *Static Analysis (SAS 2010)*, LNCS, pages 340–355. Springer, 2010. doi:10.1007/978-3-642-15769-1_21.

12 David Van Horn and Matthew Might. Abstracting abstract machines. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 51–62. ACM, 2010. doi:10.1145/1863543.1863553.

13 Brian Huffman and Ondřej Kunčar. Lifting and transfer: A modular design for quotients in Isabelle/HOL. In *Certified Programs and Proofs (CPP 2013)*, LNCS, pages 131–146. Springer, 2013. doi:10.1007/978-3-319-03545-1_9.

14 Bertrand Jeannet and Wendelin Serwe. Abstracting call-stacks for interprocedural verification of imperative programs. In *Algebraic Methodology and Software Technology (AMAST 2004)*, volume 3116 of *Lecture Notes in Computer Science*, pages 258–273. Springer, 2004. doi:10.1007/978-3-540-27815-3_22.

15 Jacques-Henri Jourdan, Vincent Laporte, Sandrine Blazy, Xavier Leroy, and David Pichardie. A formally-verified C static analyzer. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 247–259. ACM, 2015. doi:10.1145/2676726.2676966.

16 Jens Knoop and Bernhard Steffen. The interprocedural coincidence theorem. In *Compiler Construction (CC '92)*, volume 641 of *LNCS*, pages 125–140. Springer, 1992. doi:10.1007/3-540-55984-1_13.

17 Peter Lammich and Markus Müller-Olm. Formalization of conflict analysis of programs with procedures, thread creation, and monitors. *Archive of Formal Proofs*, December 2007. URL: https://isa-afp.org/entries/Program-Conflict-Analysis.html.

18 Manuel Lerchner. Voblint: A verified Goblint-style static analysis pipeline in Isabelle/HOL, 2026. doi:10.5281/zenodo.23260696.

19 Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. doi:10.1145/1538788.1538814.

20 Sébastien Michelland, Yannick Zakowski, and Laure Gonnord. Abstract interpreters: A monadic approach to modular verification. *Proceedings of the ACM on Programming Languages*, 8(ICFP), 2024. doi:10.1145/3674646.

21 Matthew Might and Panagiotis Manolios. A posteriori soundness for non-deterministic abstract interpretations. In *Verification, Model Checking, and Abstract Interpretation (VMCAI 2009)*, volume 5403 of *Lecture Notes in Computer Science*, pages 260–274. Springer, 2009. doi:10.1007/978-3-540-93900-9_22.

22 David Monniaux. Flavors of abstract interpretation. Invited tutorial, European Joint Conferences on Theory and Practice of Software (ETAPS 2024), Luxembourg, April 2024. Lecture slides, HAL hal-04750222. URL: https://hal.science/hal-04750222.

23 Tobias Nipkow. Abstract interpretation of annotated commands. In *Interactive Theorem Proving (ITP 2012)*, LNCS, pages 116–132. Springer, 2012. doi:10.1007/978-3-642-32347-8_9.

24 Tobias Nipkow and Gerwin Klein. *Concrete Semantics with Isabelle/HOL*. Springer, 2014. doi:10.1007/978-3-319-10542-0.

25 Thomas Reps, Susan Horwitz, and Mooly Sagiv. Precise interprocedural dataflow analysis via graph reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 49–61. ACM, 1995. doi:10.1145/199448.199462.

26 Xavier Rival and Laurent Mauborgne. The trace partitioning abstract domain. *ACM Transactions on Programming Languages and Systems*, 29(5), 2007. doi:10.1145/1275497.1275501.

27 Michael Schwarz. Congruences: Make % sound by restricting cases where we return a constant, 2023. Pull request 1161. URL: https://github.com/goblint/analyzer/pull/1161.

28 Michael Schwarz, Simmo Saan, Helmut Seidl, Kalmer Apinis, Julian Erhard, and Vesal Vojdani. Improving thread-modular abstract interpretation. In *Static Analysis*, volume 12913 of *LNCS*, pages 359–383, 2021. doi:10.1007/978-3-030-88806-0_18.

29 Michael Schwarz, Simmo Saan, Helmut Seidl, Julian Erhard, and Vesal Vojdani. Clustered relational thread-modular abstract interpretation with local traces. In *Programming Languages and Systems (ESOP 2023)*, volume 13990 of *LNCS*, pages 28–58, 2023. doi:10.1007/978-3-031-30044-8_2.

30 Helmut Seidl, Varmo Vene, and Markus Müller-Olm. Global invariants for analysing multithreaded applications. *Proceedings of the Estonian Academy of Sciences. Physics. Mathematics*, 52(4):413–436, 2003. doi:10.3176/phys.math.2003.4.05.

31 Helmut Seidl, Vesal Vojdani, Julian Erhard, and Michael Schwarz. Mixed flow-sensitive static analysis: Engineering modularity. In *FM 2026: Formal Methods*, pages 446–470, 2026. doi:10.1007/978-3-032-26220-2_22.

32 Micha Sharir and Amir Pnueli. Two approaches to interprocedural data flow analysis. In Steven S. Muchnick and Neil D. Jones, editors, *Program Flow Analysis: Theory and Applications*, chapter 7. Prentice-Hall, 1981.

33 Yannick Stade, Sarah Tilscher, and Helmut Seidl. The top-down solver verified: Building confidence in static analyzers. In *Computer Aided Verification*, pages 303–324, 2024. Formalization: Archive of Formal Proofs entry Top_Down_Solver, https://isa-afp.org/entries/Top_Down_Solver.html. doi:10.1007/978-3-031-65627-9_15.

34 Sarah Tilscher, Alexandra Graß, and Helmut Seidl. Verifying a solver for mixed flow-sensitive analyses. In *NASA Formal Methods (NFM 2026)*, LNCS, pages 48–69. Springer, 2026. doi:10.1007/978-3-032-28079-4_3.

35 Sarah Tilscher, Alexandra Graß, Helmut Seidl, and Yannick Stade. Proving total correctness of top-down solvers with widening and narrowing. *Journal of Automated Reasoning*, 70(2):15, 2026. doi:10.1007/s10817-026-09756-x.

36 Vesal Vojdani, Kalmer Apinis, Vootele Rõtov, Helmut Seidl, Varmo Vene, and Ralf Vogler. Static race detection for device drivers: The Goblint approach. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 391–402. ACM, 2016. doi:10.1145/2970276.2970337.

37 Daniel Wasserrab. Backing up slicing: Verifying the interprocedural two-phase Horwitz-Reps-Binkley slicer. *Archive of Formal Proofs*, November 2009. URL: https://isa-afp.org/entries/HRB-Slicing.html.